

Deep Learning Homework: Conditional GAN and Image Colorization

Mana Peiravian - 40464250

July 3, 2026

Abstract

This report describes three PyTorch experiments. The first experiment trains a Conditional GAN (cGAN) on the full Fashion-MNIST training set, using class labels to control which clothing category is generated. The second experiment trains a grayscale-to-RGB colorization autoencoder on a small natural-image dataset. This model was improved from a plain bottleneck autoencoder to a skip-connected convolutional encoder-decoder, which substantially improved the test metrics. The bonus experiment trains an RGB Adversarial Autoencoder (AAE) to study whether an image autoencoder can learn a latent space that resembles a standard Gaussian prior. All figures and metrics discussed in this report were produced by the accompanying notebooks.

1 Task 1: Conditional GAN on Fashion-MNIST

1.1 Problem Overview

A Generative Adversarial Network (GAN) contains two neural networks trained at the same time. The generator creates fake images from random noise, while the discriminator receives an image and predicts whether it is real or generated. Training is adversarial: the discriminator improves by detecting fakes, and the generator improves by producing fakes that the discriminator classifies as real.

A Conditional GAN (cGAN) adds extra information to this process. In this task, the extra information is the Fashion-MNIST class label. Instead of only asking the generator to make a realistic image, the cGAN asks it to make a realistic image of a specified class, such as a trouser, sandal, or ankle boot. The discriminator also receives the class label, so it must judge whether the image is realistic for that specific label.

1.2 Dataset and Preprocessing

The Fashion-MNIST data were found locally under `data/FashionMNIST`. The notebook directly read the raw IDX files instead of relying on `torchvision`. The training set contains 60000 grayscale images, each of size 28×28 , with 6000 examples from each of the 10 classes.

The full training set was used in the final cGAN run, so the model trained on 100% of the available training images. The test set was not required for this generative task.

The class names are: T-shirt/top, Trouser, Pullover, Dress, Coat, Sandal, Shirt, Sneaker, Bag, and Ankle boot. Each input image was converted from an integer pixel array in $[0, 255]$ to a floating-point tensor with shape $1 \times 28 \times 28$. The pixel values were normalized to $[-1, 1]$. This normalization was chosen because the generator uses a final `Tanh` activation, whose output range is also $[-1, 1]$.

1.3 Model Architecture

The generator receives two inputs: a random noise vector z and a class label y . The noise vector has dimension 100 and is sampled from a standard normal distribution. The class label is converted into a learned 50-dimensional embedding vector. The noise vector and label embedding are concatenated, giving the generator both randomness and class information. The combined vector is passed through fully connected layers with batch normalization and LeakyReLU activations. The final layer outputs 784 values, which are reshaped into a $1 \times 28 \times 28$ image and passed through `Tanh`.

The discriminator also receives two inputs: an image and the class label that the image is supposed to match. The image is flattened from $1 \times 28 \times 28$ to a 784-dimensional vector. The class label is again converted to a learned 50-dimensional embedding. These two vectors are concatenated and passed through fully connected layers with LeakyReLU activations and dropout. The discriminator outputs a single logit, not a probability. A positive logit means the discriminator leans toward “real”, and a negative logit means it leans toward “fake”.

Because the discriminator outputs logits, the loss function was `BCEWithLogitsLoss`. This is numerically more stable than applying a sigmoid manually and then using binary cross entropy.

1.4 Training Setup

Table 1: cGAN hyperparameters and final training settings.

Setting	Value
Samples used	60000 training samples (100%)
Samples per class	6000
Batch size	128
Epochs	30
Latent dimension	100
Label embedding dimension	50
Optimizer	Adam
Learning rate	0.0002
Adam betas	(0.5, 0.999)
Loss	BCEWithLogitsLoss
Training time	286.95 seconds
Final generator loss	0.8292
Final discriminator loss	0.6624

In each training iteration, the discriminator was trained on real images with correct labels and fake images generated from random labels. The generator was then trained to make the discriminator classify its generated images as real. Fixed noise and fixed labels were used to save progress snapshots at early, middle, and final training stages.

1.5 Results



Figure 1: Final cGAN samples arranged by conditioning class. Each row corresponds to one Fashion-MNIST label, and each column shows a different random noise sample for that same label.

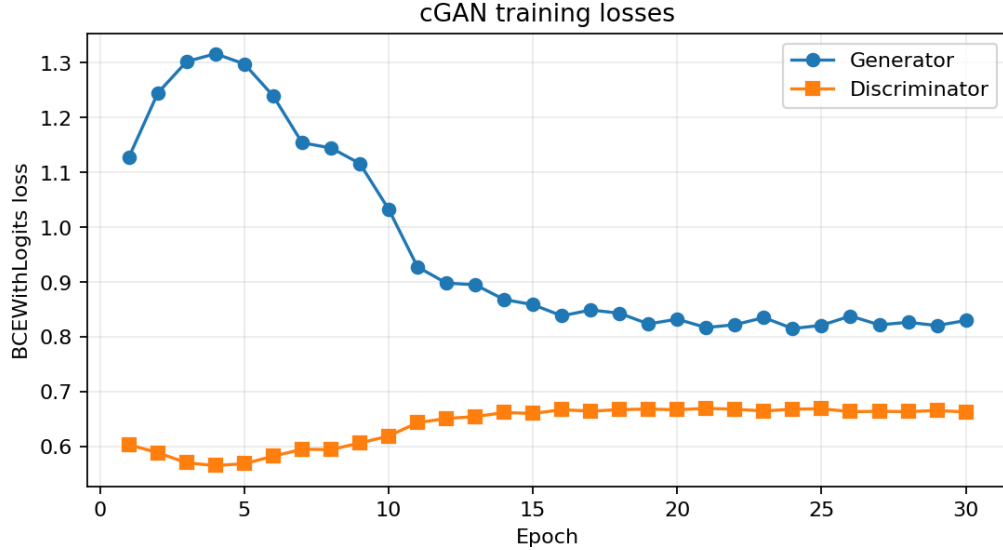


Figure 2: Generator and discriminator losses over 30 epochs on the full Fashion-MNIST training set.

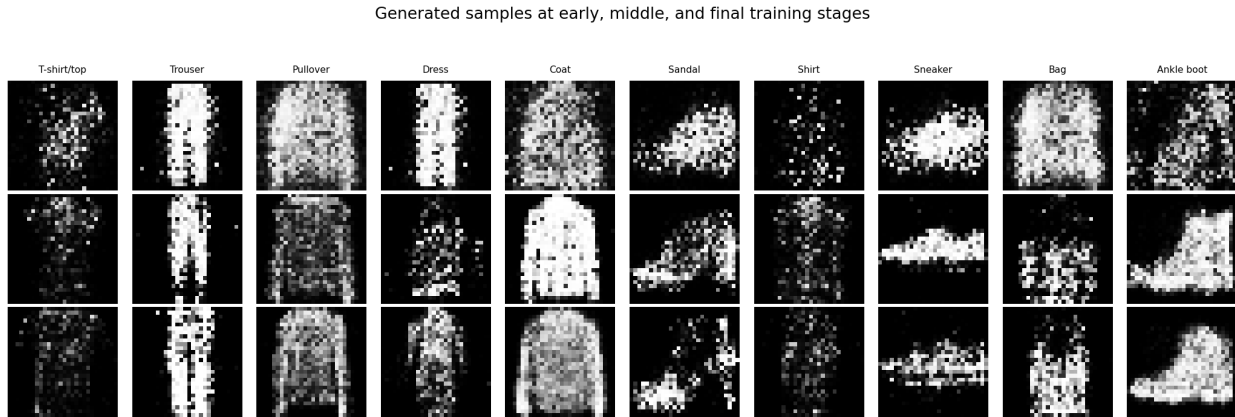


Figure 3: Generated samples at early, middle, and final training stages. The same labels and fixed noise vectors were reused so that the change over training is easier to compare.

1.6 Discussion

The full-dataset, 30-epoch run produced noticeably better samples than the earlier short run. Many generated images now have recognizable Fashion-MNIST silhouettes. Trousers, sandals, sneakers, and ankle boots are the easiest to identify because they have distinctive global shapes. Bags and several upper-body clothing classes also show recognizable structure, but they are less consistent. Pullover, coat, shirt, T-shirt/top, and dress are harder because their outlines are visually similar in low-resolution grayscale images.

The loss curves show typical adversarial behavior. The generator loss increased during the first few epochs, reaching about 1.31, meaning the discriminator was initially becoming better at detecting fake images. Later, the generator loss decreased and stabilized around

0.82–0.85. The discriminator loss began near 0.60, dipped to about 0.56, and then rose toward about 0.66. A discriminator loss near $\log(2) \approx 0.693$ often means the discriminator is less certain, but this does not automatically mean the generated images are perfect. Visual inspection is still necessary.

The progress figure and final class grid show that the conditioning signal was learned more reliably than fine visual detail. The generator usually produces the broad shape associated with the requested label, which means the label embeddings are being used. However, because the network is an MLP, each pixel is predicted through fully connected layers rather than through local convolutional filters. This makes it harder to learn clean edges, local texture, and consistent foreground boundaries. The result is a set of samples that are class recognizable but still noisy and blocky.

There are still signs of limited diversity. Some rows contain repeated prototypes, especially for classes with simple shapes. This is a mild form of mode collapse: the generator sometimes uses the label strongly but does not use the noise vector enough to create diverse variations. A likely reason is the simple MLP architecture. MLPs ignore the two-dimensional spatial structure of images, while convolutional GANs are usually better at image generation because they model local spatial patterns directly.

1.7 Answers to Required Questions

What is the main difference between a GAN and a cGAN? A standard GAN generates images from random noise only. A cGAN also receives conditioning information, such as a class label. In this task, the label tells the generator which Fashion-MNIST category to generate, and it tells the discriminator which category the image should match.

How did you provide class information to the generator and discriminator? Both networks used learned label embeddings. In the generator, the label embedding was concatenated with the random noise vector. In the discriminator, the label embedding was concatenated with the flattened image vector.

Which classes were easier or harder to generate? Trousers and footwear classes were easier because their silhouettes are distinctive. Upper-body clothing classes were harder because T-shirts, shirts, pullovers, coats, and dresses can look similar at 28×28 resolution.

How did the generated images change during training? Early images were noisy and weakly structured. By the middle and final epochs, samples developed clearer foreground-background contrast and more class-specific shapes. The final images were more recognizable than the early samples, but still pixelated and sometimes repetitive.

How did the generator and discriminator losses behave? The losses fluctuated because the two networks were trained against each other. The generator loss first increased, then decreased and stabilized. The discriminator loss eventually moved closer to 0.69, suggesting a more balanced adversarial game than at the beginning.

How could the results be improved? The most direct improvement would be to replace the MLP generator and discriminator with a convolutional DCGAN-style architecture. Other improvements include more training, stronger diversity regularization, spectral normalization, label smoothing, a smaller learning rate, and monitoring class-wise diversity to detect mode collapse.

2 Task 2: Grayscale Image Colorization using Convolutional Autoencoders

2.1 Problem Overview

Image colorization is the task of predicting an RGB image from a grayscale input. This is more difficult than ordinary reconstruction because grayscale values do not uniquely determine color. For example, two objects can have the same brightness but completely different colors. Therefore, the model must learn statistical patterns from the training data, such as which colors are common for sky, vegetation, buildings, or animals.

The model in this task is a supervised convolutional autoencoder. It receives a grayscale image as input and tries to reconstruct the original RGB image as the target. The encoder extracts features from the grayscale image, and the decoder converts those features back into a color image.

2.2 Dataset and Preprocessing

The image dataset was found under `data/AE Data`. The existing split contained 490 training images and 10 test images. The 10 test images were kept untouched for final evaluation. A deterministic 10% validation split was taken from the 490 training images, giving 441 training images and 49 validation images.

All images were loaded as RGB images, resized to 64×64 , and normalized to $[0, 1]$. The grayscale input was computed from the RGB target using the standard luminance-style weighted sum

$$Y = 0.299R + 0.587G + 0.114B.$$

Thus, the model input shape was $1 \times 64 \times 64$, and the target output shape was $3 \times 64 \times 64$.

2.3 Model Architecture

The final colorization model is a skip-connected convolutional encoder-decoder. This is still an autoencoder, but it improves on a plain bottleneck-only design by passing feature maps from the encoder directly to matching decoder stages. These skip connections help the decoder recover edges, object boundaries, and spatial layout that may be lost in a small latent bottleneck.

The encoder uses strided convolutional blocks to reduce the image resolution from 64×64 down to a compact feature representation. The decoder uses transposed convolutions to upsample the feature maps back to the original resolution. At each decoding stage, the upsampled features are concatenated with encoder features of the same spatial size. A final full-resolution grayscale skip is also concatenated before the output layer. The output layer uses a sigmoid activation so that predicted RGB values remain in $[0, 1]$.

2.4 Training Setup

Table 2: Colorization autoencoder hyperparameters and final training settings.

Setting	Value
Input size	$1 \times 64 \times 64$
Output size	$3 \times 64 \times 64$
Train/validation/test	441 / 49 / 10
Batch size	32
Maximum epochs	50
Best checkpoint epoch	42
Optimizer	Adam with weight decay 10^{-5}
Initial learning rate	0.001
Scheduler	ReduceLROnPlateau, factor 0.5, patience 4
Early-stopping patience	10 epochs
Loss	MSELoss
Best validation MSE	0.00656

MSE loss was used because the target is a normalized RGB image and the requested quantitative evaluation includes MSE and PSNR. Lower MSE means the predicted RGB pixels are closer to the target pixels on average. However, MSE can encourage conservative or slightly desaturated colors because it averages over plausible color choices.

The validation loss was monitored after each epoch. The best model checkpoint was saved according to validation MSE. Although training continued to epoch 50, the best checkpoint occurred at epoch 42 and was used for the final test evaluation.

2.5 Evaluation Metrics

The test set was evaluated using MSE, PSNR, and SSIM. MSE measures average squared pixel error, so lower is better. PSNR is derived from MSE and is measured in decibels; higher PSNR indicates smaller reconstruction error. Since images were normalized to $[0, 1]$, PSNR used $MAX_I = 1$:

$$PSNR = 20 \log_{10}(MAX_I) - 10 \log_{10}(MSE).$$

SSIM measures structural similarity between the generated image and the target image. It is useful because it considers local contrast and structure, not only pixel-wise error. Higher SSIM is better, with 1 representing perfect structural similarity.

2.6 Results

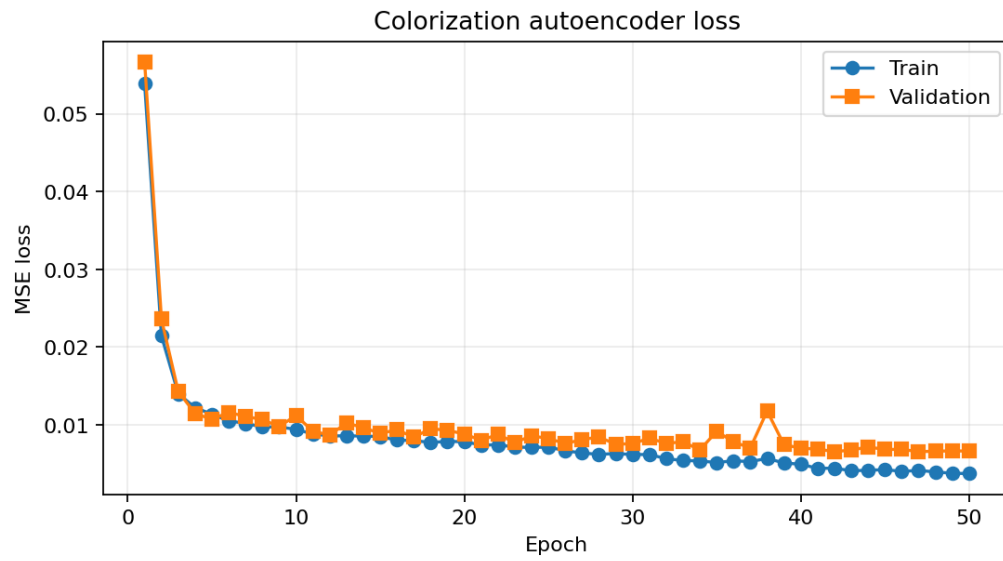


Figure 4: Training and validation loss curves for the improved skip-connected colorization autoencoder.

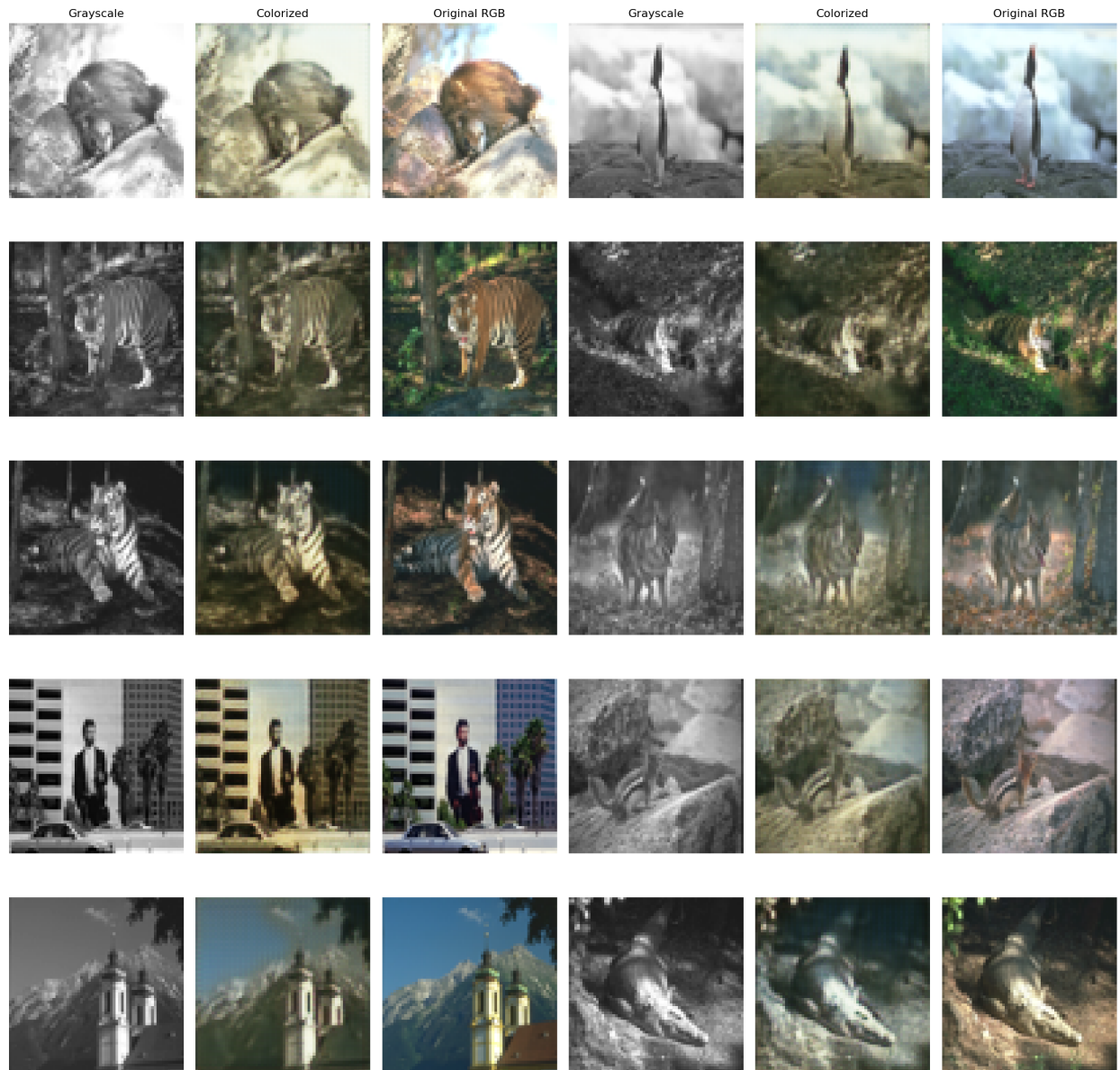


Figure 5: All 10 test images shown as grayscale input, model colorization, and RGB target.

Table 3: Colorization test metrics using the best validation checkpoint.

Image	MSE	PSNR	SSIM
105025.jpg	0.00327	24.85	0.9331
106024.jpg	0.00500	23.01	0.9444
108005.jpg	0.00192	27.16	0.9253
108070.jpg	0.00177	27.53	0.9160
108082.jpg	0.00170	27.68	0.9441
109053.jpg	0.00120	29.22	0.9241
119082.jpg	0.00748	21.26	0.9182
123074.jpg	0.00242	26.16	0.9544
126007.jpg	0.00495	23.05	0.8742
130026.jpg	0.00263	25.80	0.9427
Average	0.00324	25.57	0.9276

2.7 Discussion

The improved model performed much better than the earlier plain autoencoder baseline. The average test MSE decreased to 0.00324, the average PSNR increased to 25.57 dB, and the average SSIM reached 0.9276. The high SSIM shows that the model preserved most of the structural content of the test images. This improvement is mainly due to the skip connections, which let the decoder reuse high-resolution spatial features instead of reconstructing everything from a compressed bottleneck.

The loss curve also supports this interpretation. Most of the improvement happens in the first few epochs, when the model learns the basic grayscale-to-color mapping and image layout. After that, the training loss continues to decrease more smoothly than the validation loss. The validation curve has small oscillations and one visible spike, which is expected with only 49 validation images: a few difficult images can noticeably change the average validation MSE. Since the best validation checkpoint was saved at epoch 42 and later epochs did not substantially improve validation loss, the final evaluation used the checkpoint with the best observed generalization rather than simply the last training epoch.

The “best” and “hardest” test images depend on the metric. By pixel error, image 109053 is best, with the lowest MSE of 0.00120 and highest PSNR of 29.22 dB. By structure, image 123074 is best, with SSIM 0.9544. The weakest image by MSE and PSNR is image 119082. The weakest image by SSIM is image 126007. This is not a contradiction: MSE and PSNR penalize absolute RGB differences, while SSIM focuses more on local contrast and structure. An image can preserve edges and layout while using slightly wrong colors, producing good SSIM but worse MSE. Conversely, an image can have acceptable average color error while losing important local contrast, producing lower SSIM.

Visually, the model now preserves edges and object layout much better than before. The predicted colors are often plausible, especially for natural scenes with vegetation, sky, animals, or common outdoor tones. The model tends to produce conservative green, blue, brown, and gray color palettes because those colors are common in the dataset and because MSE rewards average color choices. Some predictions are still not perfectly color-accurate. This is expected because colorization is ambiguous: the grayscale input does not tell the

model the exact true color, and the dataset is too small for the network to learn strong object-level semantic priors.

The main limitations are the small dataset size, the modest 64×64 resolution, and the use of pixel-wise MSE loss. MSE rewards average RGB closeness, but it does not explicitly reward perceptual realism. A colorized image can have a good MSE while still looking slightly muted or semantically wrong.

2.8 Answers to Required Questions

How does the encoder-decoder structure help in image colorization? The encoder extracts increasingly abstract features from the grayscale image, such as edges, texture, and object layout. The decoder uses those features to reconstruct a three-channel RGB image. The skip connections help because they pass detailed spatial information directly from encoder layers to decoder layers.

What loss function did you use and why? MSE loss was used. It is simple, stable, and directly measures average squared RGB reconstruction error. It also matches the PSNR metric, which is computed from MSE. The drawback is that MSE can produce conservative colors when multiple colors are plausible.

Which test images were colorized better? The strongest result depends on the metric: 109053.jpg had the best MSE/PSNR, while 123074.jpg had the best SSIM. In general, images with clear structure and colors that are common in the training set were colorized better.

Which test images were more difficult to colorize? The most difficult result also depends on the metric: 119082.jpg had the largest MSE and lowest PSNR, while 126007.jpg had the lowest SSIM. Images with unusual colors, ambiguous lighting, strong texture, or less common scene content are more difficult because the grayscale input does not uniquely determine the target colors.

What are the main limitations of using a simple Convolutional Autoencoder for image colorization? Even with skip connections, the model has limited semantic understanding. It learns from only 490 training images, operates at low resolution, and uses MSE loss. These choices make it good at preserving structure but less reliable for semantically exact color choices.

How could the colorization results be improved? Possible improvements include training on a larger dataset, using higher resolution images, adding stronger data augmentation, using a deeper U-Net, using a pretrained encoder, predicting chrominance channels in LAB color space, adding perceptual loss, or adding an adversarial loss to encourage more realistic colors.

3 Bonus: Adversarial Autoencoder for Latent Space Learning

3.1 Objective

The bonus Adversarial Autoencoder (AAE) trains an RGB autoencoder while also regularizing its latent space. A normal autoencoder only learns to reconstruct inputs, so its latent vectors may have an irregular distribution. An AAE adds a latent discriminator and tries to make the encoder outputs look like samples from a chosen prior distribution. In this experiment, the chosen prior was a standard Gaussian distribution, $N(0, I)$.

3.2 Architecture

The AAE used an RGB convolutional encoder, an RGB transposed-convolution decoder, and a multilayer perceptron latent discriminator. The encoder maps a $3 \times 64 \times 64$ RGB image to a 64-dimensional latent vector. The decoder maps a 64-dimensional latent vector back to an RGB image. The latent discriminator receives a latent vector and predicts whether it came from the true Gaussian prior or from the encoder.

Table 4: Adversarial autoencoder hyperparameters and final losses.

Setting	Value
Input/output size	$3 \times 64 \times 64$
Latent dimension	64
Epochs	50
Reconstruction learning rate	0.001
Adversarial learning rate	0.0005
Prior	Standard Gaussian $N(0, I)$
Final reconstruction loss	0.01209
Final discriminator loss	0.6875
Final adversarial encoder loss	0.6755

3.3 Training Method

AAE training used three losses. First, the encoder and decoder minimized reconstruction MSE so that each RGB input image could be reconstructed. Second, the latent discriminator learned to distinguish true prior samples from encoded image vectors. Third, the encoder was trained adversarially to fool the latent discriminator, meaning encoded image vectors should look like samples from $N(0, I)$.

These objectives compete with each other. Good reconstruction encourages the latent space to store useful image information. Adversarial regularization encourages the latent vectors to follow a simple known distribution. If the regularization works well, random vectors sampled from the prior should decode into plausible images.

The final discriminator and adversarial encoder losses are close to $\log(2)$, which means the latent discriminator was no longer confidently separating prior samples from encoded samples at the end of training. This is a useful sign that the adversarial latent matching had some effect. However, matching the latent distribution does not by itself guarantee sharp or semantically meaningful images. The adversarial loss is applied only in latent space, not on decoded images, so it can make the codes look more Gaussian while the decoder still learns a blurry MSE reconstruction model.

3.4 Latent Space Analysis

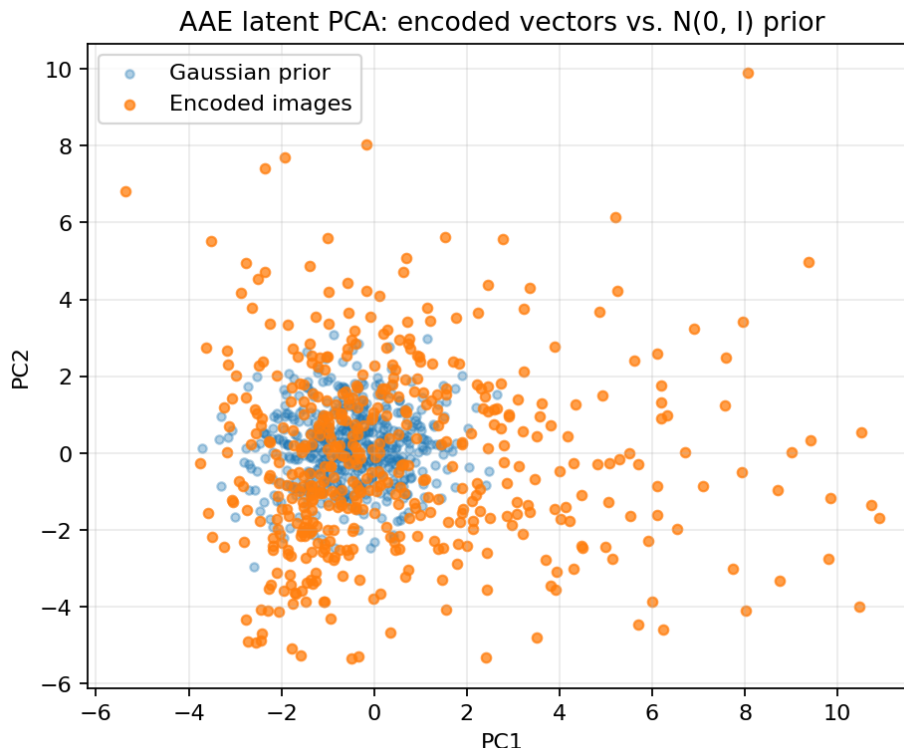


Figure 6: PCA comparison of encoded latent vectors and Gaussian prior samples.

The PCA plot compares encoded image latents with samples from the Gaussian prior after projecting both sets to two dimensions. The two distributions have a clear central overlap, so it would be inaccurate to describe them as separated. A more accurate interpretation is that the AAE achieved partial prior matching: many encoded vectors lie in the same central PCA region as the Gaussian prior samples, but the encoded vectors also have a wider spread and several outliers, especially along the positive PC1 direction and both extremes of PC2.

This matters because PCA shows only the two directions of largest variance in the combined latent set. Overlap in this two-dimensional projection suggests that adversarial regularization shaped the dominant latent directions, but it does not prove that the full 64-dimensional encoded distribution matches $N(0, I)$. The remaining spread indicates that the

encoder still uses parts of latent space in a more image-specific way than the prior. In other words, the latent matching is visible but incomplete.

3.5 Reconstruction Comparison

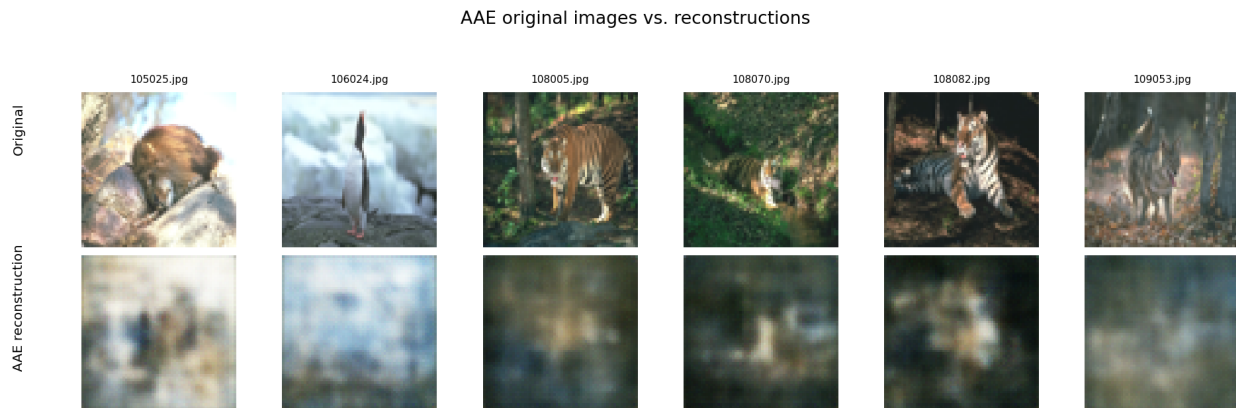


Figure 7: Original test images compared with their AAE reconstructions.

The reconstruction comparison shows that most fine image detail is already lost when a test image is compressed into the 64-dimensional AAE latent code and decoded back to RGB. This is expected because the model compresses a $3 \times 64 \times 64$ image, or 12288 pixel values, into only 64 latent values. Unlike the main colorization autoencoder, this AAE decoder has no skip connections, so high-resolution edges and textures cannot bypass the bottleneck. The MSE reconstruction loss further encourages averaged outputs when several sharp images could explain the same coarse latent representation. This explains why later latent-space visualizations are smooth and blurry: the decoder is mostly preserving coarse color and layout, not object-level structure or sharp texture.

3.6 Random Sampling

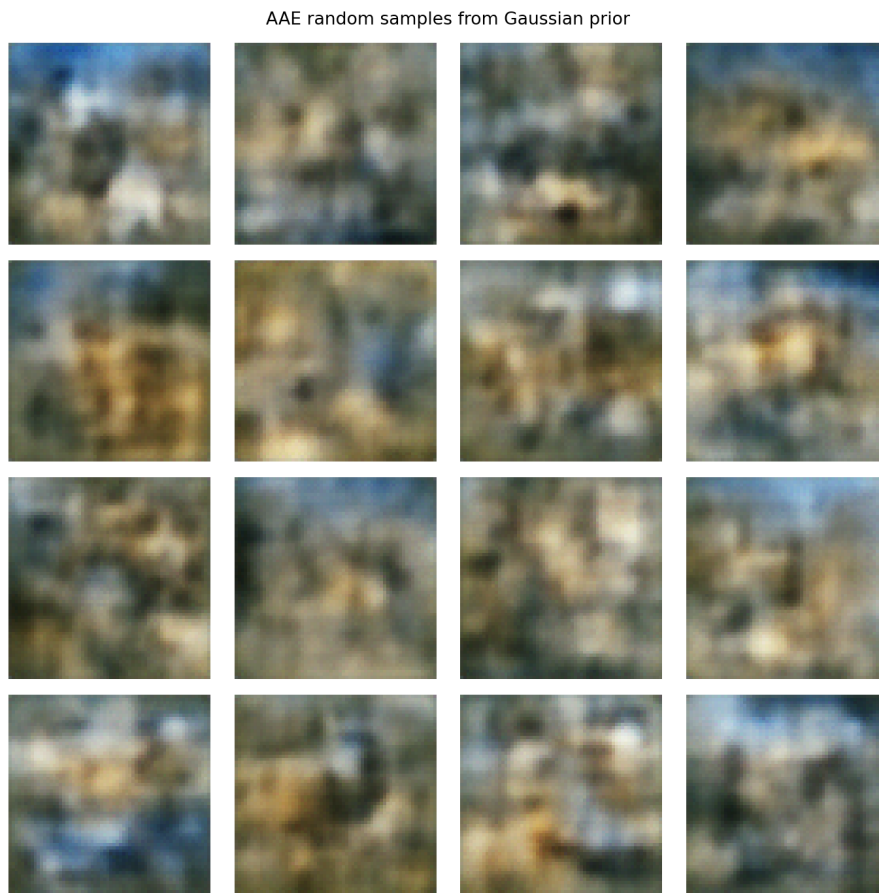


Figure 8: Images decoded from random samples drawn from the Gaussian prior.

The random samples from the prior are blurry and not semantically meaningful. This should be interpreted together with the PCA result. Since there is central overlap between encoded latents and prior samples, random Gaussian samples are not completely outside the learned latent region; this is why they decode into natural-looking color blobs rather than pure noise. However, the decoder was optimized for reconstruction from encoded image latents, not for producing sharp images from every possible prior sample. The broader encoded-latent spread and the lack of an image-level adversarial or perceptual loss mean that many prior samples decode to averaged textures and colors instead of coherent objects.

3.7 Latent Interpolation

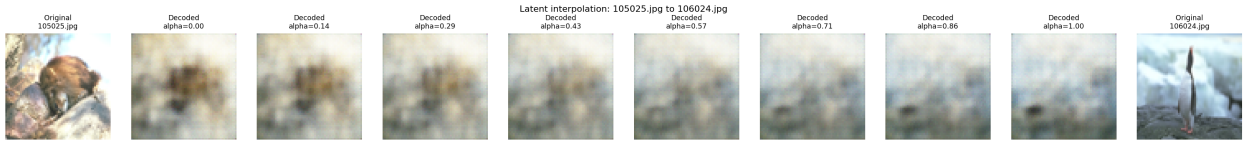


Figure 9: Original endpoint images flanking the decoded interpolation between their latent vectors.

The interpolation changes smoothly from one endpoint to the other, which indicates that the decoder is at least locally continuous in latent space. The decoded $\alpha = 0$ and $\alpha = 1$ images are reconstructions of the endpoints, not the original images themselves. Comparing those decoded endpoints against the original endpoint images makes the limitation clearer: the blur is already present before interpolation begins. The intermediate images therefore show a meaningful coarse transition in color and layout, but not a sharp semantic morph between the original animal and bird scenes. This suggests that the latent space is useful for coarse transitions but not yet strong enough for high-quality image generation.

3.8 Discussion

The AAE partially achieved its objective. It learned an RGB reconstruction model, and the latent discriminator training changed the latent-space behavior. The final discriminator and adversarial encoder losses being close to $\log(2)$ suggest that the discriminator was near chance by the end of training. This agrees with the PCA plot, where the encoded and prior samples overlap substantially in the central region. Therefore, the correct conclusion is not that prior matching completely failed; rather, prior matching was partial and mostly successful in the dominant projected directions, while the encoded distribution still had heavier spread and outliers.

The weak visual quality comes from a different limitation. The AAE regularizes the aggregate latent distribution, but the decoder is still trained with pixel-wise MSE and must reconstruct images from a very small 64-dimensional bottleneck. This encourages smooth average reconstructions. The dataset is also small, so the decoder sees limited variation and cannot learn a rich generative image manifold. Because there is no adversarial loss on the decoded images, no perceptual loss, and no skip path for details, the model has little pressure to create sharp texture or object-level realism.

More stable and higher-quality AAE results would likely require better balancing between reconstruction and adversarial losses, a stronger decoder, a larger or hierarchical latent representation, data augmentation, and possibly an image-space perceptual or adversarial loss. Longer training alone is not the main missing ingredient here, because the current run already trained for 50 epochs and the adversarial losses reached a balanced range. The bottleneck and reconstruction objective are the main reasons the visual outputs remain blurry.

4 Conclusion

The cGAN experiment shows how class labels can be used to control image generation. Training on the full Fashion-MNIST training set for 30 epochs improved the generated samples compared with the shorter baseline, but the MLP architecture still produced pixelated images and some repeated prototypes. A convolutional GAN would likely improve image quality and diversity.

The colorization experiment shows that architecture choice strongly affects reconstruction quality. The skip-connected autoencoder gave much better test results than the earlier bottleneck-only model, reaching an average MSE of 0.00324, PSNR of 25.57 dB, and SSIM of 0.9276 on the 10 test images. The model preserved structure well, although exact colors remain ambiguous.

The bonus AAE experiment demonstrated the idea of adversarial latent regularization. In this run, the prior matching was partial: the PCA projection shows substantial central overlap between encoded latents and Gaussian prior samples, and the final adversarial losses are close to a balanced discriminator game. Even so, random samples from the Gaussian prior were blurry and not semantically meaningful. This is still an informative result: it shows that matching a latent prior is not enough to learn a high-quality generative image model when the decoder is trained mainly for MSE reconstruction through a small bottleneck.